Section 6: Behavioral Design Patterns

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 66: Section Intro: Behavioral Design Patterns

SLIDE 67: Strategy: Swapping Algorithms at Runtime

SLIDE 68: Worked Example: Pricing Strategies

SLIDE 68B: Strategy Connection to Open-Closed Principle

SLIDE 69: Observer: Event-Driven Decoupling

SLIDE 70: Worked Example: Order Status Change Notifications

SLIDE 70B: Observer as Pub-Sub at the Object Level

SLIDE 70C: Interview Practice: Strategy and Observer

SLIDE 71: Command: Encapsulating Requests as Objects

SLIDE 72: Worked Example: Undo and Redo in a Text Editor

SLIDE 72B: Command for Task Queue Processing

SLIDE 73: State: Behavior Based on Internal State

SLIDE 74: Worked Example: Order Lifecycle with Valid Transitions

SLIDE 74B: State vs Strategy: When Each Applies

SLIDE 74C: Interview Practice: Command and State

SLIDE 75: Chain of Responsibility: Processing Pipelines

SLIDE 76: Worked Example: Request Validation Chain

SLIDE 77: Template Method: Algorithm Skeleton with Customizable Steps

SLIDE 77B: Worked Example: Data Processing Pipeline

SLIDE 77C: Note It Down: Behavioral Pattern Signals

SLIDE 78: Pattern Combinations in Real Designs

SLIDE 78B: Combinations: State + Observer and Decorator + Strategy

SLIDE 79: Behavioral Patterns Interview Practice and Common Mistakes

SLIDE 80: Section Summary and Cheat Sheet

SLIDE 66: Section Intro: Behavioral Design Patterns

Section 6: Behavioral Design Patterns

Behavioral patterns address how objects communicate and distribute responsibility among themselves. While structural patterns (Section 5) solved how objects are composed, behavioral patterns solve how objects interact at runtime: which object makes a decision, how events are propagated, how algorithms are selected, and how processing pipelines are organized. These patterns appear in LLD interviews more frequently than creational or structural patterns because they directly model the business logic that interviewers ask you to design.

What We Will Cover

1. Strategy: swapping algorithms at runtime
2. Observer: event-driven decoupling
3. Command: encapsulating requests as objects
4. State: objects that behave differently based on internal state
5. Chain of Responsibility: processing pipelines
6. Template Method: algorithm skeleton with customizable steps
7. Pattern combinations in real designs

Where Behavioral Patterns Appear in LLD Interviews

Interview Signal	Behavioral Pattern	Why
“Support multiple algorithms/strategies for X”	Strategy	Swap behavior without modifying calling code

Interview Signal	Behavioral Pattern	Why
“When X happens, notify Y and Z”	Observer	Decouple event producers from consumers
“Support undo/redo” or “queue operations”	Command	Encapsulate actions as first-class objects
“Object behaves differently in each state”	State	Replace complex if/else chains with state objects
“Process request through a pipeline of checks”	Chain of Responsibility	Sequential processing with optional short-circuiting
“Same algorithm structure, different steps”	Template Method	Shared skeleton, customizable sub-steps

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

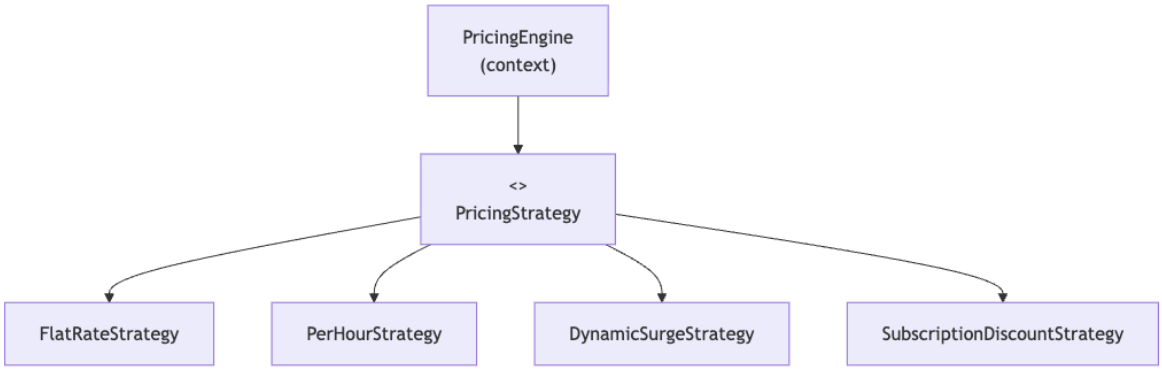
---

SLIDE 67: Strategy: Swapping Algorithms at Runtime

The Strategy pattern defines a family of algorithms, encapsulates each one behind an interface, and makes them interchangeable. The calling code (the context) delegates the algorithmic work to a strategy object and can swap strategies without any code change.

Strategy is the behavioral pattern you will use most frequently in LLD interviews. It is the direct implementation of the Open-Closed Principle (Section 3): adding a new algorithm means writing a new class, not modifying existing code.

The Structure



The `PricingEngine` holds a reference to a `PricingStrategy` interface. It does not know or care which algorithm is behind the interface. The strategy can be set at construction time, loaded from configuration, or even changed mid-request based on business rules.

When to Use Strategy

Use Strategy when you see any of these signals in the requirements:

1. "Support multiple ways to do X" (pricing, sorting, compression, routing, validation)
2. An existing if/else or switch statement that selects behavior based on a type or configuration value
3. An algorithm that may change independently of the code that uses it
4. Different customers, tiers, or contexts need different behavior for the same operation

Interview Relevance: Strategy is the most commonly tested behavioral pattern. When you hear "support multiple types of X" in an LLD interview, Strategy should be your first instinct. State this explicitly: "Since we need multiple pricing strategies, I will define a PricingStrategy interface."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 68: Worked Example: Pricing Strategies

Consider a ride-sharing platform like Uber that needs to support multiple pricing models. The pricing model depends on the city, the time of day, the rider's subscription tier, and current demand.

The Interface

```
public interface PricingStrategy {
    Money calculateFare(RideRequest request);
    String getStrategyName();
}
```

Three Implementations

```
public class FlatRateStrategy implements PricingStrategy {
    private final Money baseFare;
    private final Money perKilometerRate;

    public FlatRateStrategy(Money baseFare, Money perKilometerRate) {
        this.baseFare = baseFare;
        this.perKilometerRate = perKilometerRate;
    }

    @Override
    public Money calculateFare(RideRequest request) {
        double distance = request.getDistanceKm();
        return baseFare.add(perKilometerRate.multiplyBy(distance));
    }

    @Override
    public String getStrategyName() {
        return "FLAT_RATE";
    }
}
```

```
public class DynamicSurgeStrategy implements PricingStrategy
{
    private final PricingStrategy baseStrategy;
    private final DemandTracker demandTracker;

    public DynamicSurgeStrategy(PricingStrategy baseStrategy,
                                DemandTracker demandTracker)
{
        this.baseStrategy = baseStrategy;
        this.demandTracker = demandTracker;
    }

    @Override
    public Money calculateFare(RideRequest request) {
        Money baseFare = baseStrategy.calculateFare(request);
        double surgeMultiplier = demandTracker.getSurgeMultip
lier(request.getPickupZone());
        return baseFare.multiplyBy(surgeMultiplier);
    }

    @Override
    public String getStrategyName() {
        return "DYNAMIC_SURGE";
    }
}
```

```
public class SubscriptionDiscountStrategy implements PricingS
trategy {
    private final PricingStrategy baseStrategy;
    private final double discountPercentage;

    public SubscriptionDiscountStrategy(PricingStrategy baseS
trategy,
                                        double discountPerce
```

```
ntage) {
        this.baseStrategy = baseStrategy;
        this.discountPercentage = discountPercentage;
    }

    @Override
    public Money calculateFare(RideRequest request) {
        Money baseFare = baseStrategy.calculateFare(request);
        return baseFare.multiplyBy(1.0 - discountPercentage);
    }

    @Override
    public String getStrategyName() {
        return "SUBSCRIPTION_DISCOUNT";
    }
}
```

The Context

```
public class PricingEngine {
    private final PricingStrategyResolver strategyResolver;

    public PricingEngine(PricingStrategyResolver strategyReso
lver) {
        this.strategyResolver = strategyResolver;
    }

    public FareEstimate calculateFare(RideRequest request) {
        PricingStrategy strategy = strategyResolver.resolve(r
equest);
        Money fare = strategy.calculateFare(request);
        return new FareEstimate(fare, strategy.getStrategyNam
e());
    }
}
```

Notice that `DynamicSurgeStrategy` and `SubscriptionDiscountStrategy` both wrap another `PricingStrategy`. This is the Strategy + Decorator combination: the base strategy calculates the fare, and the wrapping strategies modify it. This composability is a direct benefit of programming to interfaces.

Interview Relevance: This example demonstrates three things interviewers evaluate: the interface extraction, the composability of strategies (wrapping one strategy with another), and the separation of strategy resolution from strategy execution.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 68B: Strategy Connection to Open-Closed Principle

Strategy is OCP (Section 3, Slide 29) made concrete. The connection is direct and worth stating explicitly in interviews.

Without Strategy (OCP Violation)

```
public Money calculateFare(RideRequest request, String pricin
gType) {
    if (pricingType.equals("FLAT_RATE")) {
        return baseFare.add(perKmRate.multiplyBy(request.getD
istanceKm()));
    } else if (pricingType.equals("PER_MINUTE")) {
        return baseFare.add(perMinuteRate.multiplyBy(request.
getDurationMinutes()));
    } else if (pricingType.equals("SURGE")) {
        double multiplier = demandTracker.getSurgeMultiplier
(request.getPickupZone());
        return baseFare.add(perKmRate.multiplyBy(request.getD
istanceKm()))
                       .multiplyBy(multiplier);
    }
    throw new IllegalArgumentException("Unknown pricing type:
```

```
" + pricingType);
}
```

Adding a new pricing type means editing this method. The class is modified for every new business requirement. This is the OCP violation we identified in Section 3.

With Strategy (OCP Compliant)

Adding a new pricing type means writing one new class that implements `PricingStrategy`. The `PricingEngine`, the `PricingStrategyResolver`, and all existing strategies are untouched.

The Mapping

SOLID Principle	Strategy Pattern Equivalent
"Open for extension"	Write a new `PricingStrategy` implementation
"Closed for modification"	`PricingEngine` is never edited
Interface abstraction	`PricingStrategy` interface defines the contract
Polymorphic dispatch	`strategy.calculateFare(request)` calls the right implementation

Interview Tip: In an LLD interview, when you extract an if/else chain into a Strategy pattern, explicitly state: "This follows the Open-Closed Principle. Adding a new pricing type requires one new class. No existing code is modified." Connecting the pattern to the SOLID principle shows the interviewer that you understand the why, not just the how.

Interview Relevance: Interviewers often present a switch statement and ask you to refactor it. The refactoring is Strategy. Naming the OCP connection demonstrates depth beyond pattern mechanics.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 69: Observer: Event-Driven Decoupling

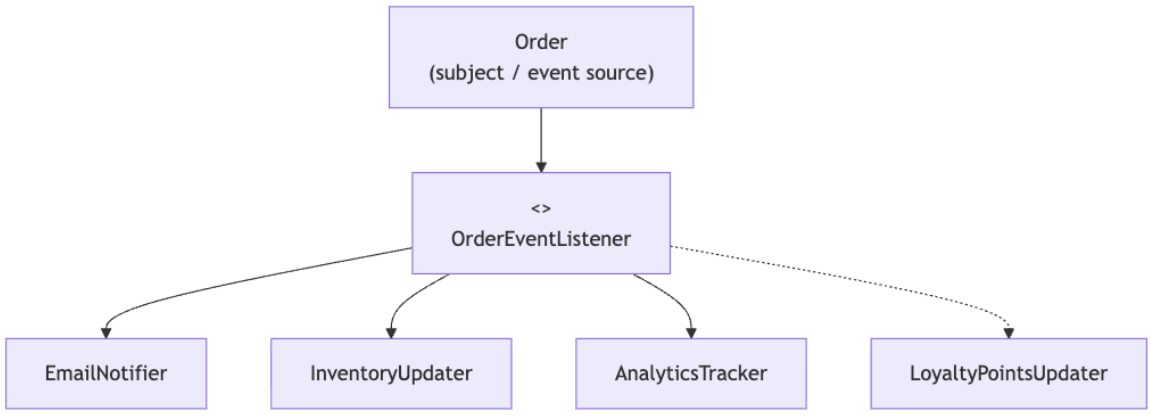
The Observer pattern defines a one-to-many dependency between objects so that when one object (the subject) changes state, all its dependents (the observers) are notified automatically. The subject does not know what the observers do with the notification. It just fires the event.

The Core Problem Observer Solves

Consider an order management system. When an order's status changes to "shipped," the system needs to: send an email to the customer, update the inventory count, log the event for analytics, and notify the warehouse management system. Without Observer, the `Order` class must know about all four of these systems and call them directly. Adding a fifth action (update a loyalty points tracker) requires modifying the `Order` class.

With Observer, the `Order` fires a "status changed" event. Any number of listeners can subscribe to that event. Adding the loyalty points tracker means registering a new listener. The `Order` class never changes.

The Structure



The dashed line to `LoyaltyPointsUpdater` represents a future listener added without modifying the `Order` class. This is OCP applied to event handling.

Observer vs Direct Method Calls

Dimension	Direct Calls	Observer
Coupling	Order knows about every listener	Order knows only the listener interface
Adding a new reaction	Modify the Order class	Register a new listener
Removing a reaction	Modify the Order class	Unregister the listener
Testing Order in isolation	Must mock all listeners	Can test with zero listeners
Runtime flexibility	Fixed at compile time	Listeners added/removed at runtime

Interview Relevance: Observer appears in any LLD question where an event in one part of the system triggers reactions in other parts. The signal phrase is "when X happens, Y and Z should also happen." If Y and Z are in different domains, Observer is the answer.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 70: Worked Example: Order Status Change Notifications

The Event and Listener Interface

```
public class OrderEvent {
    private final String orderId;
    private final OrderStatus previousStatus;
    private final OrderStatus newStatus;
    private final Instant timestamp;

    public OrderEvent(String orderId, OrderStatus previousSta
tus,
                      OrderStatus newStatus) {
        this.orderId = orderId;
        this.previousStatus = previousStatus;
        this.newStatus = newStatus;
        this.timestamp = Instant.now();
```

```
    }

    public String getOrderId() { return orderId; }
    public OrderStatus getPreviousStatus() { return previousS
tatus; }
    public OrderStatus getNewStatus() { return newStatus; }
    public Instant getTimestamp() { return timestamp; }
}
```

```
public interface OrderEventListener {
    void onOrderStatusChanged(OrderEvent event);
}
```

The Subject (Order)

```
public class Order {
    private final String orderId;
    private OrderStatus status;
    private final List<OrderEventListener> listeners = new Ar
rayList<>();

    public Order(String orderId) {
        this.orderId = orderId;
        this.status = OrderStatus.CREATED;
    }

    public void addListener(OrderEventListener listener) {
        listeners.add(listener);
    }

    public void removeListener(OrderEventListener listener) {
        listeners.remove(listener);
    }

    public void updateStatus(OrderStatus newStatus) {
```

```
        OrderStatus previousStatus = this.status;
        this.status = newStatus;
        OrderEvent event = new OrderEvent(orderId, previousSt
atus, newStatus);
        notifyListeners(event);
    }

    private void notifyListeners(OrderEvent event) {
        for (OrderEventListener listener : listeners) {
            listener.onOrderStatusChanged(event);
        }
    }
}
```

Two Listeners

```
public class EmailNotifier implements OrderEventListener {
    private final EmailService emailService;
    private final CustomerRepository customerRepository;

    public EmailNotifier(EmailService emailService,
                         CustomerRepository customerRepositor
y) {
        this.emailService = emailService;
        this.customerRepository = customerRepository;
    }

    @Override
    public void onOrderStatusChanged(OrderEvent event) {
        if (event.getNewStatus() == OrderStatus.SHIPPED) {
            Customer customer = customerRepository.findByOrde
rId(event.getOrderId());
            emailService.send(customer.getEmail(), "Your orde
r has shipped",
                buildShippingEmailBody(event));
        }
```

```
    }
}
```

```
public class AnalyticsTracker implements OrderEventListener {
    private final AnalyticsClient analyticsClient;

    public AnalyticsTracker(AnalyticsClient analyticsClient)
{
        this.analyticsClient = analyticsClient;
    }

    @Override
    public void onOrderStatusChanged(OrderEvent event) {
        analyticsClient.track("order_status_change", Map.of(
            "order_id", event.getOrderId(),
            "from_status", event.getPreviousStatus().name(),
            "to_status", event.getNewStatus().name(),
            "timestamp", event.getTimestamp().toString()
        ));
    }
}
```

Each listener handles one concern. The `Order` class does not know that emails are sent or that analytics are tracked. It fires events and moves on.

Interview Relevance: This exact structure (event object, listener interface, subject with listener list, notify loop) is the standard Observer implementation. Showing it with a realistic domain (order management) rather than a toy example signals production experience.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 70B: Observer as Pub-Sub at the Object Level

Observer at the class level is the same concept as publish-subscribe (pub-sub) at the system level. Understanding this connection helps in both LLD and system design interviews.

The Mapping

Observer (Class Level)	Pub-Sub (System Level)
Subject (Order)	Publisher (Order Service)
Observer interface	Message schema / event contract
Listener list	Message broker (Kafka, RabbitMQ, SNS)
`notifyListeners()`	`broker.publish(event)`
`listener.onEvent()`	Consumer processes message from queue
In-process, synchronous	Cross-service, asynchronous

When to Use Object-Level Observer vs System-Level Pub-Sub

Consideration	Object-Level Observer	System-Level Pub-Sub
Listeners are in the same process	Yes	Overkill
Listeners are in different services	Not possible	Required
Need guaranteed delivery	No (if listener throws, event is lost)	Yes (message broker retries)
Need ordering guarantees	Simple (loop order)	Complex (partitioning, sequencing)
Latency requirement	Microseconds	Milliseconds to seconds
Complexity	Low (30 lines of code)	High (broker infra, serialization, consumer groups)

In an LLD interview, you are designing within a single service, so object-level Observer is appropriate. In a system design interview, the same concept scales to pub-sub with a message broker.

Interview Tip: When you use Observer in an LLD interview, mention: “At the class level, I am using Observer for in-process event notification. If this system scales to multiple services, this pattern maps directly to a pub-sub architecture with a

message broker." This bridge between LLD and system design impresses interviewers because it shows you think across abstraction levels.

Interview Relevance: The Observer-to-pub-sub connection is tested when interviewers ask "how would this scale?" Having the mapping ready lets you answer immediately.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 70C: Interview Practice: Strategy and Observer

Scenario

You are designing a content moderation system for a platform like YouTube. User-uploaded content must be checked against multiple moderation strategies (profanity filter, spam detection, copyright check, nudity detection). Each strategy produces a moderation decision (approve, flag for review, reject). When content is rejected, the system should notify the content creator, log the decision for compliance, and update the creator's trust score.

The interviewer asks: "How would you structure this?"

Pause the video and identify which patterns apply for 60 seconds before continuing.

Weak Answer

"I would create a `ContentModerator` class with methods for each check type and if/else logic for notifications."

This puts all moderation logic and all notification logic in one class, violating SRP. Adding a new moderation strategy or a new notification type requires modifying the same class.

Strong Answer

"The multiple moderation strategies are a Strategy problem. I would define a `ModerationStrategy` interface with a `moderate(Content)` method. Each check (profanity, spam, copyright, nudity) is a separate implementation. A `ModerationPipeline` iterates through all registered strategies and aggregates their decisions.

The notification reactions (notify creator, log for compliance, update trust score) are an Observer problem. The `ModerationPipeline` fires a `ModerationDecisionEvent` when content is rejected. Three listeners handle the notifications independently.

Adding a new moderation check means writing one new Strategy. Adding a new notification means registering one new Observer. Neither change modifies existing code."

SLIDE 71: Command: Encapsulating Requests as Objects

The Command pattern turns a request or operation into a standalone object that contains all the information needed to perform the action. Instead of calling a method directly, you create a Command object that represents the method call, and pass that object around. The Command can be stored, queued, logged, or undone.

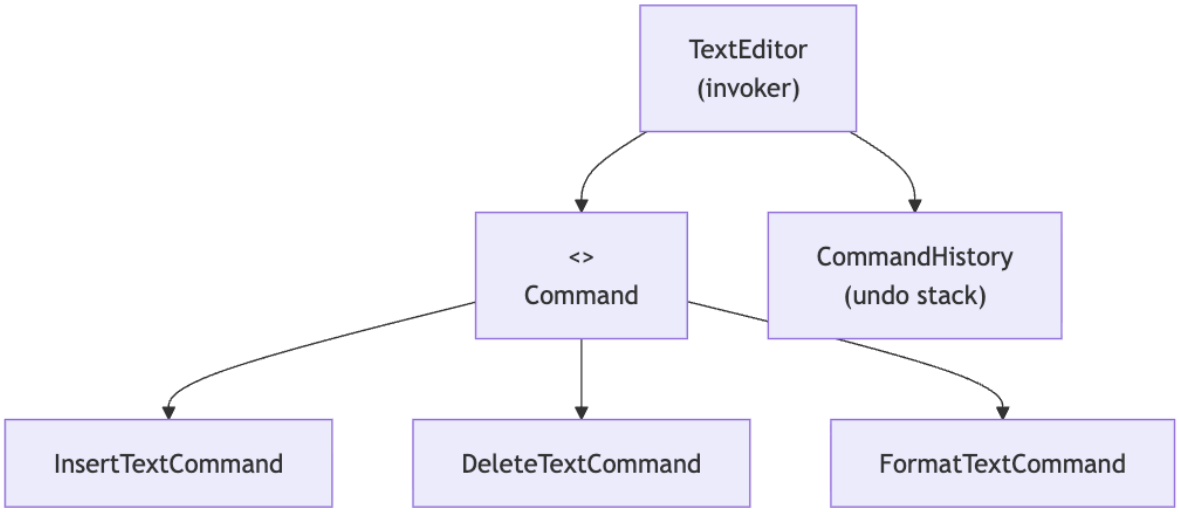
When to Use Command

Command is appropriate when you need to:

1. **Support undo/redo:** Each command knows how to execute itself and how to reverse itself
2. **Queue operations:** Commands are stored in a queue and executed later (task processing, job scheduling)
3. **Log operations:** Commands are persisted for audit trails or replay
4. **Batch operations:** Multiple commands are collected and executed as a group

The unifying idea is that the action becomes a first-class object rather than just a method call. Once it is an object, you can do everything with it that you can do with any other object: store it, pass it, serialize it, undo it.

The Structure



The `TextEditor` (invoker) does not perform text operations directly. It creates Command objects and executes them. Each executed command is pushed onto the `CommandHistory` stack. Undoing pops the last command and calls its `undo()` method.

Interview Relevance: Command appears in LLD interviews for text editors, drawing applications, task schedulers, and any system that needs undo/redo or operation queuing. The signal phrase is “support undo” or “queue operations for later execution.”

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 72: Worked Example: Undo and Redo in a Text Editor

The Command Interface

```
public interface TextCommand {
    void execute();
    void undo();
    String getDescription();
}
```

Two Concrete Commands

```
public class InsertTextCommand implements TextCommand {
    private final TextDocument document;
    private final int position;
    private final String textToInsert;

    public InsertTextCommand(TextDocument document, int posit
ion,
                             String textToInsert) {
        this.document = document;
        this.position = position;
        this.textToInsert = textToInsert;
    }

    @Override
    public void execute() {
        document.insertAt(position, textToInsert);
    }

    @Override
    public void undo() {
        document.deleteRange(position, position + textToInser
t.length());
    }

    @Override
    public String getDescription() {
        return "Insert '" + textToInsert + "' at position " +
position;
    }
}
```

```
public class DeleteTextCommand implements TextCommand {
    private final TextDocument document;
```

```
    private final int startPosition;
    private final int endPosition;
    private String deletedText;

    public DeleteTextCommand(TextDocument document, int start
Position,
                             int endPosition) {
        this.document = document;
        this.startPosition = startPosition;
        this.endPosition = endPosition;
    }

    @Override
    public void execute() {
        deletedText = document.getTextRange(startPosition, en
dPosition);
        document.deleteRange(startPosition, endPosition);
    }

    @Override
    public void undo() {
        document.insertAt(startPosition, deletedText);
    }

    @Override
    public String getDescription() {
        return "Delete text from " + startPosition + " to " +
endPosition;
    }
}
```

The Command History (Undo/Redo Stack)

```
public class CommandHistory {
    private final Deque<TextCommand> undoStack = new ArrayDeq
ue<>();
```

```
    private final Deque<TextCommand> redoStack = new ArrayDeq
ue<>();

    public void executeCommand(TextCommand command) {
        command.execute();
        undoStack.push(command);
        redoStack.clear();
    }

    public void undo() {
        if (undoStack.isEmpty()) {
            throw new IllegalStateException("Nothing to und
o");
        }
        TextCommand command = undoStack.pop();
        command.undo();
        redoStack.push(command);
    }

    public void redo() {
        if (redoStack.isEmpty()) {
            throw new IllegalStateException("Nothing to red
o");
        }
        TextCommand command = redoStack.pop();
        command.execute();
        undoStack.push(command);
    }
}
```

The critical detail: `DeleteTextCommand` saves the deleted text in `execute()` so that `undo()` can restore it. Each command captures everything it needs to reverse itself. The redo stack is cleared whenever a new command is executed, because a new action after an undo invalidates the redo history.

Interview Relevance: The undo/redo text editor is one of the most popular Command pattern interview questions. The interviewer evaluates whether you capture enough state in each command to support reversal, and whether your history stack handles the undo-then-new-action edge case (clearing redo).

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 72B: Command for Task Queue Processing

Command is not limited to undo/redo. In production systems, Command is frequently used to queue operations for deferred or asynchronous execution.

The Scenario

A platform like Shopify needs to process post-order tasks: generate an invoice PDF, update inventory across warehouses, and notify the shipping provider. These tasks should be queued and processed asynchronously rather than blocking the order confirmation response.

The Command Interface for Queuing

```
public interface TaskCommand {
    void execute();
    String getTaskType();
    int getMaxRetries();
}
```

A Concrete Task

```
public class GenerateInvoiceCommand implements TaskCommand {
    private final String orderId;
    private final InvoiceGenerator invoiceGenerator;
    private final DocumentStore documentStore;

    public GenerateInvoiceCommand(String orderId,
                                  InvoiceGenerator invoiceGe
```

```
nerator,
                                    DocumentStore documentStor
e) {
        this.orderId = orderId;
        this.invoiceGenerator = invoiceGenerator;
        this.documentStore = documentStore;
    }

    @Override
    public void execute() {
        byte[] invoicePdf = invoiceGenerator.generate(orderI
d);
        documentStore.store(orderId + "-invoice.pdf", invoice
Pdf);
    }

    @Override
    public String getTaskType() {
        return "GENERATE_INVOICE";
    }

    @Override
    public int getMaxRetries() {
        return 3;
    }
}
```

The Task Queue Processor

```
public class TaskQueueProcessor {
    private final BlockingQueue<TaskCommand> taskQueue;

    public TaskQueueProcessor(BlockingQueue<TaskCommand> task
Queue) {
        this.taskQueue = taskQueue;
    }
```

```
    public void enqueue(TaskCommand task) {
        taskQueue.offer(task);
    }

    public void processNext() {
        TaskCommand task = taskQueue.poll();
        if (task == null) {
            return;
        }
        int attempts = 0;
        while (attempts < task.getMaxRetries()) {
            try {
                task.execute();
                return;
            } catch (Exception e) {
                attempts++;
                if (attempts >= task.getMaxRetries()) {
                    handleFailedTask(task, e);
                }
            }
        }
    }
}
```

The order confirmation endpoint creates three Command objects (`GenerateInvoiceCommand`, `UpdateInventoryCommand`, `NotifyShippingCommand`), enqueues them, and returns immediately. The processor handles execution, retries, and failure recovery separately.

Interview Tip: When an interviewer asks about asynchronous processing or background jobs, Command is the pattern that encapsulates the work. Say: "Each task is a Command object. The queue stores commands and the processor executes them with retry logic. Adding a new post-order task means creating one new Command class and enqueueing it."

Interview Relevance: Task queue processing appears in e-commerce, notification systems, and data pipeline interviews. Command + Queue is the standard answer.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 73: State: Behavior Based on Internal State

The State pattern allows an object to change its behavior when its internal state changes. Instead of using a series of if/else or switch statements to check the current state and decide what to do, you encapsulate each state's behavior in a separate class. The object delegates to its current state object, and transitions between states are explicit.

The Problem State Solves

Consider an order in an e-commerce system. An order moves through states: CREATED, CONFIRMED, SHIPPED, DELIVERED, CANCELLED. The behavior of the order depends on its current state:

- In CREATED state, the order can be confirmed or cancelled
- In CONFIRMED state, the order can be shipped or cancelled (with a refund)
- In SHIPPED state, the order can be delivered but cannot be cancelled
- In DELIVERED state, no further transitions are allowed

Without State Pattern: The If/Else Nightmare

```
public void cancel(Order order) {
    if (order.getStatus().equals("CREATED")) {
        order.setStatus("CANCELLED");
    } else if (order.getStatus().equals("CONFIRMED")) {
        order.setStatus("CANCELLED");
        paymentService.refund(order.getTransactionId());
    } else if (order.getStatus().equals("SHIPPED")) {
        throw new IllegalStateException("Cannot cancel a ship
ped order");
    } else if (order.getStatus().equals("DELIVERED")) {
```

```
            throw new IllegalStateException("Cannot cancel a deli
vered order");
        }
    }
```

Every method that operates on the order (`confirm()`, `ship()`, `cancel()`, `deliver()`) has a similar if/else chain. Adding a new state (RETURNED) requires modifying every method. The behavior for each state is scattered across the entire class.

With the State pattern, all behavior for the CONFIRMED state lives in one `ConfirmedState` class. All behavior for the SHIPPED state lives in one `ShippedState` class. Adding a RETURNED state means writing one new class.

Interview Relevance: State appears in any LLD interview involving lifecycle management: order processing, document workflows, game character behavior, or connection management. The signal phrase is "object behaves differently depending on its state."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 74: Worked Example: Order Lifecycle with Valid Transitions

The State Interface

```
public interface OrderState {
    void confirm(OrderContext context);
    void ship(OrderContext context);
    void deliver(OrderContext context);
    void cancel(OrderContext context);
    String getStateName();
}
```

The Context (Order)

```
public class OrderContext {
    private OrderState currentState;
```

```
    private final String orderId;
    private final PaymentService paymentService;
    private String transactionId;

    public OrderContext(String orderId, PaymentService paymen
tService) {
        this.orderId = orderId;
        this.paymentService = paymentService;
        this.currentState = new CreatedState();
    }

    public void setState(OrderState newState) {
        this.currentState = newState;
    }

    public void confirm(String transactionId) {
        this.transactionId = transactionId;
        currentState.confirm(this);
    }

    public void ship() { currentState.ship(this); }
    public void deliver() { currentState.deliver(this); }
    public void cancel() { currentState.cancel(this); }

    public PaymentService getPaymentService() { return paymen
tService; }
    public String getTransactionId() { return transactionId;
}
    public String getOrderId() { return orderId; }
}
```

Two State Implementations

```
public class CreatedState implements OrderState {
    @Override
    public void confirm(OrderContext context) {
```

```
        context.setState(new ConfirmedState());
    }

    @Override
    public void ship(OrderContext context) {
        throw new InvalidTransitionException("Cannot ship an
unconfirmed order");
    }

    @Override
    public void deliver(OrderContext context) {
        throw new InvalidTransitionException("Cannot deliver
an unconfirmed order");
    }

    @Override
    public void cancel(OrderContext context) {
        context.setState(new CancelledState());
    }

    @Override
    public String getStateName() { return "CREATED"; }
}
```

```
public class ConfirmedState implements OrderState {
    @Override
    public void confirm(OrderContext context) {
        throw new InvalidTransitionException("Order is alread
y confirmed");
    }

    @Override
    public void ship(OrderContext context) {
        context.setState(new ShippedState());
    }
```

```
    @Override
    public void deliver(OrderContext context) {
        throw new InvalidTransitionException("Cannot deliver
before shipping");
    }

    @Override
    public void cancel(OrderContext context) {
        context.getPaymentService().refund(context.getTransac
tionId());
        context.setState(new CancelledState());
    }

    @Override
    public String getStateName() { return "CONFIRMED"; }
}
```

Each state class defines exactly which transitions are valid from that state and what side effects (like refunds) accompany each transition. Invalid transitions throw exceptions with clear messages. Adding a RETURNED state means writing a `ReturnedState` class and adding a `return()` method to the interface. Existing states simply throw `InvalidTransitionException` for `return()` unless the transition is valid from their state.

Interview Relevance: The State pattern with valid transition guards is the standard answer for order lifecycle LLD questions. Interviewers evaluate whether you handle invalid transitions explicitly rather than silently ignoring them.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 74B: State vs Strategy: When Each Applies

State and Strategy have identical structure: an interface, multiple implementations, and a context that delegates to the current implementation. The difference is in

intent and lifecycle.

The Key Distinction

Dimension	Strategy	State
Intent	Choose one algorithm from a set	Object changes behavior as internal state changes
Who selects the implementation	External code or configuration	The current state determines the next state
Transitions	Typically set once, rarely changes	Transitions happen throughout the object's lifetime
Awareness of other implementations	Strategies do not know about each other	Each state knows which states it can transition to
Example	Pricing algorithm chosen based on customer tier	Order moves through Created, Confirmed, Shipped
Replaces	If/else selecting an algorithm	If/else checking current state

The Simple Test

Ask: "Does the object change its behavior over time?"

If yes, it is State. The order starts as CREATED, transitions to CONFIRMED, then SHIPPED. The behavior changes as the object progresses through its lifecycle.

If no, it is Strategy. The pricing algorithm is selected once based on configuration and does not change during the request.

Common Interview Mistake

Candidates confuse the two because the UML diagrams look identical. The way to differentiate in an interview: if the object transitions between implementations during its lifetime and each implementation knows which other implementations it can transition to, it is State. If the implementation is selected externally and stays fixed, it is Strategy.

Interview Tip: If an interviewer asks "is this State or Strategy?", answer with the transition test: "In State, the current implementation determines the next implementation. In Strategy, external code selects the implementation. Here, the order's current state determines valid next states, so this is State."

Interview Relevance: The State-vs-Strategy distinction is a direct interview question at mid-to-senior levels. Having the transition test ready demonstrates precision in your design vocabulary.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 74C: Interview Practice: Command and State

Scenario

You are designing a drawing application like Figma. The application supports multiple tools (pen, eraser, shape tool, text tool). Each tool behaves differently when the user clicks on the canvas. The application also needs full undo/redo support for all drawing operations.

The interviewer asks: "How would you structure the tool behavior and the undo system?"

Pause the video and identify which patterns apply for 60 seconds before continuing.

Weak Answer

"I would use if/else to check the current tool and create a list of operations for undo."

This conflates two separate concerns. The tool behavior selection and the undo system are different problems that need different patterns.

Strong Answer

"The tool behavior that changes based on which tool is selected is a State problem. A `ToolState` interface defines `onCanvasClick(Point)`, `onCanvasDrag(Point, Point)`, and `onCanvasRelease(Point)`. Each tool (pen, eraser, shape, text) is a separate State implementation. When the user switches tools, the application transitions to the corresponding state.

The undo/redo system is a Command problem. Each drawing action creates a Command object (`DrawLineCommand`, `EraseAreaCommand`, `InsertShapeCommand`) that captures

all information needed to execute and reverse the action. A `CommandHistory` manages the undo and redo stacks.

The two patterns work together: the current `ToolState` creates the appropriate `Command` when the user interacts with the canvas, and the `CommandHistory` manages the lifecycle of those commands."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 75: Chain of Responsibility: Processing Pipelines

The Chain of Responsibility pattern passes a request along a chain of handlers. Each handler decides either to process the request, pass it to the next handler, or reject it. The request moves through the chain until a handler fully processes it or the chain ends.

When to Use Chain of Responsibility

Chain of Responsibility is appropriate when:

1. Multiple checks or transformations must be applied to a request in sequence
2. The set of checks can vary (some requests skip certain checks)
3. Each check is independent and has a single responsibility
4. The chain can short-circuit (a failed check stops further processing)

Real-World Analogy

HTTP middleware in web frameworks is Chain of Responsibility. A request passes through authentication, authorization, rate limiting, input validation, and logging middleware. Each middleware can process the request and pass it forward, or reject it and return an error response.

The Structure



Each handler has a reference to the next handler in the chain. If a handler cannot process the request (authentication fails, rate limit exceeded), it short-circuits and returns an error. If the handler succeeds, it passes the request to the next handler.

Interview Relevance: Chain of Responsibility appears in LLD interviews for request processing, validation pipelines, approval workflows, and middleware design. The signal phrase is “process this request through a series of checks.”

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 76: Worked Example: Request Validation Chain

The Handler Interface

```
public interface RequestHandler {
    HandlerResult handle(ApiRequest request);
    void setNext(RequestHandler nextHandler);
}
```

Abstract Base Handler

```
public abstract class AbstractRequestHandler implements Reque
stHandler {
    private RequestHandler nextHandler;

    @Override
    public void setNext(RequestHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    protected HandlerResult passToNext(ApiRequest request) {
        if (nextHandler == null) {
            return HandlerResult.success();
        }
        return nextHandler.handle(request);
```

```
    }
}
```

Concrete Handlers

```
public class AuthenticationHandler extends AbstractRequestHan
dler {
    private final TokenValidator tokenValidator;

    public AuthenticationHandler(TokenValidator tokenValidato
r) {
        this.tokenValidator = tokenValidator;
    }

    @Override
    public HandlerResult handle(ApiRequest request) {
        String token = request.getHeader("Authorization");
        if (token == null || token.isBlank()) {
            return HandlerResult.rejected(401, "Missing authe
ntication token");
        }
        TokenValidationResult validation = tokenValidator.val
idate(token);
        if (!validation.isValid()) {
            return HandlerResult.rejected(401, "Invalid or ex
pired token");
        }
        request.setAuthenticatedUser(validation.getUserId());
        return passToNext(request);
    }
}
```

```
public class RateLimitHandler extends AbstractRequestHandler
{
    private final RateLimiter rateLimiter;
```

```
    public RateLimitHandler(RateLimiter rateLimiter) {
        this.rateLimiter = rateLimiter;
    }

    @Override
    public HandlerResult handle(ApiRequest request) {
        String userId = request.getAuthenticatedUser();
        if (!rateLimiter.allowRequest(userId)) {
            return HandlerResult.rejected(429, "Rate limit ex
ceeded");
        }
        return passToNext(request);
    }
}
```

```
public class InputValidationHandler extends AbstractRequestHa
ndler {
    private final RequestValidator requestValidator;

    public InputValidationHandler(RequestValidator requestVal
idator) {
        this.requestValidator = requestValidator;
    }

    @Override
    public HandlerResult handle(ApiRequest request) {
        ValidationResult validation = requestValidator.valida
te(request.getBody());
        if (!validation.isValid()) {
            return HandlerResult.rejected(400, validation.get
ErrorMessage());
        }
        return passToNext(request);
```

```
    }
}
```

Assembling the Chain

```
AuthenticationHandler authHandler = new AuthenticationHandler
(tokenValidator);
RateLimitHandler rateLimitHandler = new RateLimitHandler(rate
Limiter);
InputValidationHandler validationHandler = new InputValidatio
nHandler(requestValidator);

authHandler.setNext(rateLimitHandler);
rateLimitHandler.setNext(validationHandler);

HandlerResult result = authHandler.handle(incomingRequest);
```

Each handler checks one concern. If authentication fails, the request never reaches rate limiting. If rate limiting rejects the request, validation is skipped. Each handler is independently testable and independently reusable across different API endpoints.

Interview Relevance: The request validation chain is the most common Chain of Responsibility example in interviews. Showing explicit short-circuiting (rejected requests skip remaining handlers) is the key detail interviewers look for.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 77: Template Method: Algorithm Skeleton with Customizable Steps

The Template Method pattern defines the skeleton of an algorithm in a base class, with specific steps deferred to subclasses. The algorithm's overall structure is fixed, but the implementation of individual steps varies.

When to Use Template Method

Template Method is appropriate when:

1. Multiple classes share the same algorithmic structure but differ in specific steps
2. You want to enforce a consistent sequence of operations while allowing customization of individual operations
3. The "what happens and in what order" is fixed, but "how each step is performed" varies

The Structure

```
public abstract class DataImportPipeline {
    public final ImportResult runImport(String sourcePath) {
        RawData rawData = readFromSource(sourcePath);
        List<Record> parsedRecords = parse(rawData);
        List<Record> validRecords = validate(parsedRecords);
        List<Record> transformedRecords = transform(validReco
rds);
        int savedCount = saveToDatabase(transformedRecords);
        return new ImportResult(parsedRecords.size(), validRe
cords.size(), savedCount);
    }

    protected abstract RawData readFromSource(String sourcePa
th);
    protected abstract List<Record> parse(RawData rawData);

    protected List<Record> validate(List<Record> records) {
        return records.stream()
            .filter(Record::isValid)
            .collect(Collectors.toList());
    }

    protected abstract List<Record> transform(List<Record> re
cords);
```

```
    protected int saveToDatabase(List<Record> records) {
        return recordRepository.batchInsert(records);
    }
}
```

The `runImport()` method is `final`, which means subclasses cannot change the algorithm's sequence. Subclasses must implement `readFromSource()`, `parse()`, and `transform()`. They can optionally override `validate()` and `saveToDatabase()` if the default behavior is not appropriate.

Template Method vs Strategy

Dimension	Template Method	Strategy
Customization mechanism	Inheritance (override methods)	Composition (inject interface)
Algorithm structure	Fixed in the base class	Entirely up to the strategy
Coupling	Subclass is coupled to the base class	Strategy is independent
Flexibility	Compile-time customization	Runtime customization
When to use	Algorithm structure is shared, steps vary	Entire algorithm varies

Interview Relevance: Template Method appears in LLD interviews for data processing pipelines, report generation, and ETL workflows. The signal phrase is "the steps are always the same, but the implementation of each step varies by data type."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 77B: Worked Example: Data Processing Pipeline

Three Data Import Implementations

```
public class CsvImportPipeline extends DataImportPipeline {
    @Override
```

```
    protected RawData readFromSource(String sourcePath) {
        return FileReader.readFile(sourcePath);
    }

    @Override
    protected List<Record> parse(RawData rawData) {
        return CsvParser.parse(rawData.getContent(), ",");
    }

    @Override
    protected List<Record> transform(List<Record> records) {
        return records.stream()
            .map(record -> record.trimFields())
            .map(record -> record.normalizeDate("yyyy-MM-d
d"))
            .collect(Collectors.toList());
    }
}
```

```
public class JsonApiImportPipeline extends DataImportPipeline
{
    private final HttpClient httpClient;

    public JsonApiImportPipeline(HttpClient httpClient) {
        this.httpClient = httpClient;
    }

    @Override
    protected RawData readFromSource(String sourcePath) {
        HttpResponse response = httpClient.get(sourcePath);
        return new RawData(response.getBody());
    }

    @Override
    protected List<Record> parse(RawData rawData) {
```

```
        JsonArray jsonArray = JsonParser.parseArray(rawData.g
etContent());
        return jsonArray.stream()
            .map(JsonMapper::toRecord)
            .collect(Collectors.toList());
    }

    @Override
    protected List<Record> transform(List<Record> records) {
        return records.stream()
            .map(record -> record.flattenNestedFields())
            .map(record -> record.convertUnits("metric"))
            .collect(Collectors.toList());
    }
}
```

Both pipelines follow the same sequence: read, parse, validate, transform, save. The steps are different (CSV reads a file while JSON fetches from an API; CSV splits by comma while JSON maps from JSON objects), but the overall algorithm is identical. Adding a new data source (XML, Parquet, database dump) means writing one new subclass that implements the three abstract methods.

When Template Method Becomes Problematic

Template Method uses inheritance, which means it inherits the fragile base class risk from Section 2. If the base class `DataImportPipeline` changes the order of steps or adds a new required step, all subclasses may break. For systems where the pipeline steps themselves need to be swapped at runtime, Strategy (or Chain of Responsibility) is a better fit.

Interview Tip: When you use Template Method in an interview, mention the inheritance trade-off proactively: “I am using Template Method here because the algorithm structure is fixed and shared. If the pipeline steps needed to vary at runtime, I would use Strategy or Chain of Responsibility instead.” This shows the interviewer that you made a deliberate trade-off.

Interview Relevance: Template Method is less commonly tested than Strategy or Observer, but it appears in data pipeline and ETL interview questions. Knowing

when to use it (fixed structure, varying steps) versus Strategy (entire algorithm varies) is the key differentiator.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 77C: Note It Down: Behavioral Pattern Signals

These are the signals that tell you which behavioral pattern to reach for during an LLD interview. Memorize the mapping from requirement to pattern.

Signal-to-Pattern Quick Reference

Signal in the Requirements	Pattern	One-Line Justification
"Support multiple algorithms for X"	Strategy	Interface per algorithm, swap at runtime
"When X happens, notify Y, Z, and W"	Observer	Subject fires events, listeners react independently
"Support undo/redo"	Command	Each action is an object that knows how to reverse itself
"Queue operations for later execution"	Command	Commands stored in queue, executed by processor
"Object behaves differently in each state"	State	Each state is a class, transitions are explicit
"Process request through a series of checks"	Chain of Responsibility	Each handler checks one thing, passes or rejects
"Same algorithm structure, different step implementations"	Template Method	Base class owns the sequence, subclasses customize steps
"Replace this growing switch statement"	Strategy (or State)	Switch on type = Strategy. Switch on state = State
"Add reactions to events without modifying the source"	Observer	New listener, no changes to the event source

Three Questions to Ask Yourself

1. **Is the variation in the algorithm or in the object's lifecycle?** Algorithm = Strategy. Lifecycle = State.

2. **Does the action need to be stored, queued, or reversed?** If yes, Command.
3. **Is the processing sequential with possible short-circuits?** If yes, Chain of Responsibility.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 78: Pattern Combinations in Real Designs

Real systems rarely use a single design pattern. Most LLD interview solutions combine two or three patterns, each solving a different aspect of the problem. Recognizing which patterns to combine, and how they interact, is what separates strong LLD answers from textbook recitations.

Combination 1: Strategy + Factory

The Factory creates the right Strategy based on configuration or input. This separates the "which algorithm to use" decision from the "how the algorithm works" implementation.

```
public class PricingStrategyFactory {
    private final Map<String, PricingStrategy> strategies;

    public PricingStrategyFactory(Map<String, PricingStrategy
> strategies) {
        this.strategies = strategies;
    }

    public PricingStrategy resolveStrategy(RideRequest reques
t) {
        String tier = request.getCustomerTier();
        PricingStrategy strategy = strategies.get(tier);
        if (strategy == null) {
            return strategies.get("DEFAULT");
        }
        return strategy;
```

```
    }
}
```

The Factory owns the selection logic. The Strategy owns the calculation logic. Neither knows about the other's internals.

Combination 2: Observer + Command

Events are fired as Observer notifications, and each event handler creates and enqueues a Command for deferred execution. This decouples event detection from event processing and supports retry logic.

```
public class OrderShippedListener implements OrderEventListen
er {
    private final TaskQueue taskQueue;

    public OrderShippedListener(TaskQueue taskQueue) {
        this.taskQueue = taskQueue;
    }

    @Override
    public void onOrderStatusChanged(OrderEvent event) {
        if (event.getNewStatus() == OrderStatus.SHIPPED) {
            taskQueue.enqueue(new SendShippingEmailCommand(ev
ent.getOrderId()));
            taskQueue.enqueue(new UpdateInventoryCommand(even
t.getOrderId()));
            taskQueue.enqueue(new NotifyWarehouseCommand(even
t.getOrderId()));
        }
    }
}
```

The Observer detects the event synchronously. The Commands handle the work asynchronously with retry support.

Interview Relevance: Interviewers expect combined patterns in strong LLD answers. Mentioning "I would combine Strategy and Factory here" signals that you

think in composable building blocks rather than applying one pattern to the entire problem.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 78B: Combinations: State + Observer and Decorator + Strategy

Combination 3: State + Observer

State transitions fire Observer events. This separates “what happens during a transition” from “who reacts to the transition.”

```
public class ConfirmedState implements OrderState {
    @Override
    public void ship(OrderContext context) {
        context.setState(new ShippedState());
        context.fireEvent(new OrderEvent(
            context.getOrderId(),
            OrderStatus.CONFIRMED,
            OrderStatus.SHIPPED
        ));
    }
}
```

The State class handles the transition logic and state change. The Observer listeners handle reactions (send email, update analytics, notify warehouse). Adding a new reaction to shipping does not require modifying any State class.

Combination 4: Decorator + Strategy

Decorators wrap a Strategy to add cross-cutting concerns (logging, timing, caching) to any algorithm.

```
public class TimedPricingStrategy implements PricingStrategy
{
```

```
    private final PricingStrategy delegate;
    private final MetricsRecorder metrics;

    public TimedPricingStrategy(PricingStrategy delegate,
                                MetricsRecorder metrics) {
        this.delegate = delegate;
        this.metrics = metrics;
    }

    @Override
    public Money calculateFare(RideRequest request) {
        long start = System.nanoTime();
        Money result = delegate.calculateFare(request);
        long duration = System.nanoTime() - start;
        metrics.recordTiming("pricing_calculation", duratio
n);
        return result;
    }

    @Override
    public String getStrategyName() {
        return delegate.getStrategyName();
    }
}
```

Any pricing strategy can be wrapped with timing. The timing decorator does not know or care which algorithm it wraps. The algorithm does not know it is being timed.

Common Pattern Combinations in LLD Interviews

Combination	When It Appears
Strategy + Factory	Selection logic separated from algorithm logic
Observer + Command	Event detection triggers deferred/retryable work
State + Observer	State transitions fire events for decoupled reactions
Decorator + Strategy	Cross-cutting concerns added to any algorithm

Combination	When It Appears
Chain of Responsibility + Factory	Pipeline handlers created dynamically per request type
Template Method + Strategy	Fixed pipeline with interchangeable individual steps

Interview Tip: When your LLD design uses two or more patterns, name them explicitly: “I am combining Observer for event notification with Command for deferred execution.” This tells the interviewer you are composing patterns deliberately, not accidentally arriving at a complex structure.

Interview Relevance: Pattern combinations are where senior-level LLD answers distinguish themselves. Interviewers evaluate whether you can identify the right combination for the problem rather than forcing one pattern to solve everything.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 79: Behavioral Patterns Interview Practice and Common Mistakes

Practice Scenario

You are designing a payment processing workflow for a platform like Stripe. The workflow must: validate the payment request, check for fraud, select the appropriate payment method (credit card, bank transfer, digital wallet), process the payment, and handle post-payment actions (send receipt, update ledger, trigger webhook to merchant). Fraud detection uses multiple strategies that can be swapped. Post-payment actions should not block the payment response. The payment itself transitions through states (initiated, processing, completed, failed, refunded).

The interviewer asks: “Walk me through your class design.”

Pause the video and identify which patterns apply for 90 seconds before continuing.

Key Design Decisions

The validation pipeline (validate, fraud check) is Chain of Responsibility. The fraud detection strategies (rule-based, ML-based, manual review) are Strategy. The

payment method selection is Factory. The payment lifecycle (initiated, processing, completed, failed, refunded) is State. The post-payment actions (receipt, ledger, webhook) are Observer + Command (Observer fires synchronously, Command objects are queued for async processing).

A strong answer identifies five patterns and explains where each applies. Each pattern solves one problem. No pattern is forced to solve a problem outside its purpose.

Common Behavioral Pattern Mistakes

Mistake	Why It Is Wrong	Stronger Approach
Using Strategy when the object changes behavior over time	Strategy is for fixed algorithm selection, not lifecycle transitions	Use State for objects that transition between behaviors
Observer with synchronous heavy processing in listeners	Slow listeners block the subject and each other	Use Observer + Command to defer heavy work to a queue
Command without enough state to undo	Undo requires the exact data that was changed	Capture all affected data in the command before executing
State with transitions handled by the context	The context becomes a switch statement over states	Each State class handles its own valid transitions
Chain of Responsibility with handler order dependencies	Reordering handlers breaks the pipeline	Document ordering requirements and test edge cases
Using every pattern in one design	Over-engineering creates complexity without solving real problems	Use only the patterns that solve specific identified problems

Interview Relevance: The “payment processing workflow” combines the highest number of patterns in a single realistic scenario. Practicing this end-to-end scenario builds your ability to compose patterns under interview pressure.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 80: Section Summary and Cheat Sheet

Concepts Covered

Pattern	One-Line Summary
Strategy	Define a family of algorithms behind an interface, swap them at runtime
Observer	One-to-many event notification, decoupling subject from listeners
Command	Encapsulate actions as objects for undo, queuing, or logging
State	Replace if/else on state with state objects that handle their own behavior
Chain of Responsibility	Pass a request through a pipeline of handlers with short-circuiting
Template Method	Fixed algorithm skeleton in base class, customizable steps in subclasses
Strategy + Factory	Factory selects algorithm, Strategy executes it
Observer + Command	Observer detects events, Commands defer processing
State + Observer	State transitions fire events for decoupled reactions
Decorator + Strategy	Cross-cutting concerns added to any algorithm transparently

Common Interview Mistakes

Mistake	Stronger Approach
“I used Strategy for the order lifecycle”	“The order transitions between states over time, so I used State. Strategy is for selecting an algorithm that stays fixed.”
“I put all notification logic in the Order class”	“I used Observer so the Order fires events and listeners handle notifications independently.”
“I stored undo history as strings”	“Each action is a Command object with execute() and undo() methods that capture all necessary state.”
“I put all validation in one handler”	“I used Chain of Responsibility with separate handlers for authentication, authorization, rate limiting, and input validation.”
“I combined Strategy, Observer, Command, State, Chain of Responsibility,	“I used Strategy and Observer for this specific problem. Patterns solve specific problems, not

Mistake	Stronger Approach
and Template Method in one class"	every problem."

Key Points to Remember

Pattern	Recognition Signal	Key Question
Strategy	"Multiple algorithms for X"	"Does the algorithm need to be swappable?"
Observer	"When X happens, notify Y and Z"	"Are the listeners in different domains?"
Command	"Undo, queue, or log operations"	"Does the action need to be a first-class object?"
State	"Behavior changes based on state"	"Does the object transition through a lifecycle?"
Chain of Responsibility	"Pipeline of checks"	"Can each check short-circuit the pipeline?"
Template Method	"Same structure, different steps"	"Is the algorithm skeleton shared across types?"

How This Connects to Other Sections

- **Section 2 (OOP):** Strategy relies on polymorphism and composition. State replaces if/else chains with polymorphic dispatch. Observer decouples through interfaces.
- **Section 3 (SOLID):** Strategy implements OCP. Observer enables DIP (subject depends on listener interface). Command applies SRP (each command encapsulates one action). Chain of Responsibility applies SRP to request processing.
- **Section 4 (Creational):** Factory Method creates Strategy objects. Builder constructs complex Command objects. Singleton (cautiously) manages event buses.
- **Section 5 (Structural):** Decorator wraps Strategy objects with cross-cutting concerns. Adapter bridges external observers. Facade coordinates multiple observers behind a simple API.

- **Section 7 (Pattern Selection):** The signal-to-pattern mapping for behavioral patterns is the densest part of the selection framework because behavioral patterns are tested most frequently in LLD interviews.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn